# Clase 34 — BGP y el enrutamiento entre operadores

**Bloque:** D (Capa de red) · **Semana:** 12 · **Duración estimada:** 115 min **Correlativa:** Clase 33 (link-state, Dijkstra y OSPF) · **Sigue con:** Clase 35 (fragmentación, MTU y path MTU) **Material de referencia:** Tanenbaum cap. 5 (§5.6.4 BGP, "A Case Study in Internet Routing") · RFC 4271, RFC 4456, RFC 5396, RFC 6793, RFC 1997, RFC 2385, RFC 9234, RFC 6810, RFC 6811

---

## 🎯 Al terminar esta clase vas a poder

1. Explicar por qué Internet **no es un grafo único** y por qué en el nivel inter-AS hace falta **política** y no solo un camino más corto.
2. Definir un **Sistema Autónomo (AS)** y un **ASN**, explicar cómo se asignan y por qué hoy son **32 bits**.
3. Describir **BGP-4**: la sesión entre vecinos, el **handshake** completo y la máquina de estados.
4. Explicar por qué BGP es **path-vector** y no distance-vector, y por qué la detección de loops con `AS_PATH` es **exacta** y no probabilística.
5. Recitar la **tabla de desempates** de atributos de decisión en el orden correcto y aplicarla a un caso con cuatro rutas candidatas.
6. Distinguir **eBGP** de **iBGP**, explicar la regla del **full-mesh iBGP** y qué problema resuelve **Route Reflection**.
7. Explicar el **"hot potato"** con números, y cómo lo corrige el atributo **MED**.
8. Describir la **seguridad de BGP**: por qué TCP no alcanza, qué son **RPKI/ROA**, qué son los **roles BGP** y por qué los **route leaks** son el fallo más común.

> **Prerequisito del examen:** `2009-03 Pregunta 2` (10 pts — enrutamiento jerárquico, IGP con jerarquías y **hot-potato routing**), `2001-08 Problema 2` (30 pts — enrutamiento jerárquico y tablas de ruteo), `eirc0702 Problema 2` (apartado e — enrutamiento hacia Internet con rutas por defecto), `2007-03 Problema 2a` (IGP con jerarquías y la frontera interior/exterior), `2006-08 P3` (distance-vector: el contraste con BGP). Todos se resuelven en §3.

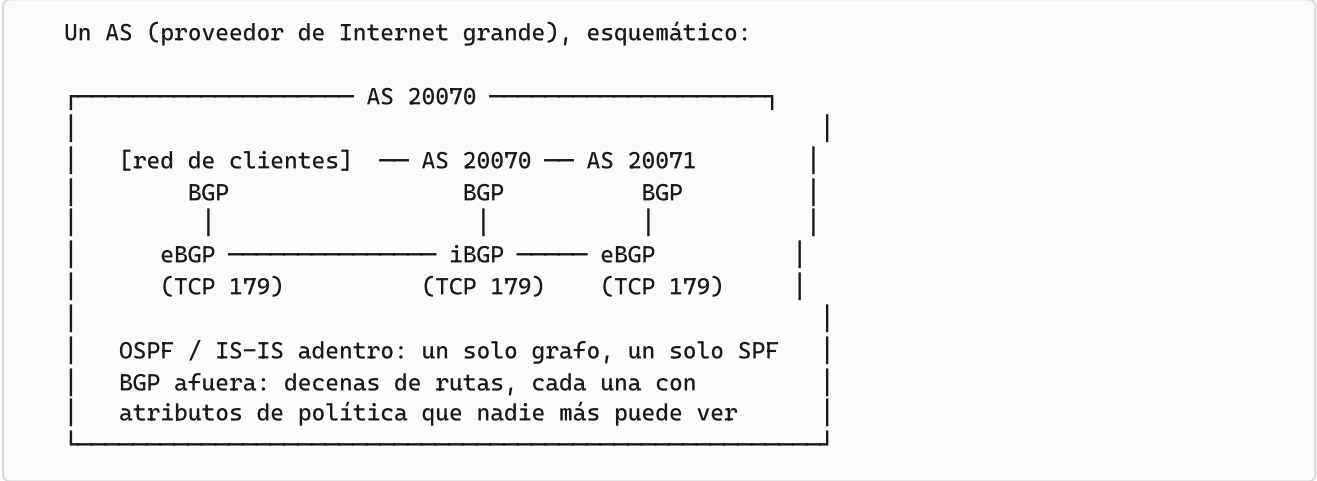
---

# PARTE 1 — TEORÍA

## 1.1 El problema: Internet no es una red de redes con un grafo, sino con una política

La clase anterior terminó con un OSPF corriendo Dijkstra sobre **un** grafo. Todo estaba bien porque había **un solo dominio administrativo**: un área, un LSDB, un árbol.

Internet no es eso. Internet es **decenas de miles de dominios administrativos independientes**, cada uno con sus equipos, su política, sus acuerdos comerciales y su dinero. Cada uno decide **a quién le anuncia qué**.




**El número.** En el momento en que se da esta clase, el número de sistemas autónomos visibles en la tabla global de BGP ronda los **setenta mil**. Ese número cambia todos los meses, así que **medilo vivo** en la página de stats de APNIC o en Cloudflare Radar antes de citar un número, y decí siempre **qué** estás contando (AS activos en la tabla global, no "empresas que tienen internet"). En el curso de 2011 la cifra era de unos treinta mil: **ha más que duplicado**, y esa es la razón por la que BGP no escala como los protocolos que usamos adentro.

> ⚠️ **Trampa conceptual clásica.** Decir que "Internet es una red de redes" es correcto pero inútil, igual que decir "una LAN es una red de redes de una sola máquina". Lo que hace especial a Internet es que cada red de redes tiene **una política distinta**, y esa política es lo que se negocia.

### Por qué no alcanza con "el camino más corto"

```
En un grafo, la pregunta es:        ¿cuál es el camino más corto?
  → hay UNA respuesta
  → el algoritmo la calcula

En Internet, la pregunta es:         ¿cuál de los 300 caminos
                                       que tengo hasta tu red
                                       quiero usar HOY?
  → hay TRESCIENTAS respuestas válidas
  → y todas dan un resultado
  → la elección es un ACTO DE POLÍTICA, no un cálculo
```

Las tres razones por las que el camino más corto **no** es la respuesta correcta en el nivel inter-AS:

| Razón | Consecuencia práctica |
|---|---|
| **Coste económico** | El enlace tránsit **se paga**. Mandar por el más corto puede es exactamente lo que nadie quiere. |
| **Relaciones comerciales** | Un AS transita de other's traffic solo si tiene peering o acuerdo. |
| **Latencia y riesgo** | El camino más corto en número de saltos puede pasar por un enlace transatlántico de 3 saltos. |

> **La frase para el examen:** *"BGP no es el algoritmo más corto de Internet: es el protocolo que permite a cada AS expresar su propia política de alcanzabilidad, y por eso siempre encuentra una respuesta, aunque no sea la de menor costo."*

## 1.2 El Sistema Autónomo y el ASN

Un **sistema autónomo (AS, Autonomous System)** es **un conjunto de prefijos IP y un conjunto de routers que los anuncia con una política única y consistente**. Es una noción **administrativa**, no técnica.

```
¿Qué hace que un conjunto de prefijos sea UN AS?
─────────────────────────────────────────────────

· un solo Routing Policy declarado
· un único ORIGIN de las rutas en BGP
· routing autónomo: decide qué anuncia y a quién,
  y sobre qué NumericRoutesMinúsculas puede contar de forma razonable
· un ASN que lo identifica en el plano de control

(lo que NO define un AS: la tecnología de los enlaces,
 el IGP que usen adentro, o el número de routers)
```

| Cómo se asigna | Detalle |
|---|---|
| **RIR → ISP** | Los cinco RIR (ARIN, RIPE NCC, APNIC, LACNIC, AFRINIC) asignan bloques **públicos** de IPv4 y /32 de IPv6 |
| **ISP grande → cliente** | Un operador que ya tiene un AS puede delegar un **propio ASN** a un cliente grande. El cliente es un AS **independiente** |
| **Multihoming** | Un cliente con **dos** proveedores tiene su propio ASN y dos sesiones eBGP, y elige qué se anuncia a quién |
| **Privado** | Hay rangos reservados para uso **privado**: se usan en redes grandes que no tocan Internet |

## El ASN: de 16 a 32 bits

```
AS de 16 bits (2 bytes)   ── el clásico
──────────────────────────────────────

Rango 1 .. 65535
65535 valores. Agotado a fines de los '90.

AS de 32 bits (4 bytes)   ── el vigente
──────────────────────────────────────

Rango 1 .. 4294967295
65536 valores nuevos, más los 65535 viejos reasignados.

Notación decimal "asdot":   1.10   o  1.0.0.10
Notación decimal "asplain":  65546 (o 1.0.0.10 en_configs)
```

| Documento | Qué dice |
|---|---|
| `RFC 4893` | El **problema**: el campo de 2 octetos del AS_PATH ya no alcanza |
| `RFC 6793` | El **mecanismo**: se agrega una **capacidad opcional** (Capability TLV) al mensaje OPEN que dice "soporte AS de 4 octetos", y un atributo opcional con el AS de 32 bits |
| `RFC 5396` | La **notación decimal** ( `asdot` y `asplain` ) |

**El detalle fino, que es lo que se pregunta:** los dos routers que se hablan **tienen que ponerse de acuerdo**. El router que anuncia un ASN mayor que 65535 tiene que announce la **Capacidad** en el OPEN. Si el vecino es viejo y no entiende la capacidad, la sesión **no se establece**: no hay un modo degradado. Por eso la transición del AS de 16 al de 32 bits tardó años, y no fue un problema de falta de números sino de **interoperabilidad del handshake**.

> **No inventar:** `AS_SET` (cuando una ruta atraviesa un AS que no anuncia prefijo, se agrupa en un AS_SET) y el tema de los **private ASNs** existen, pero el número de RFC que reserva los rangos privados **no está** en nuestra lista de referencia: verificar en `rfc-editor.org` antes de citarlo en un examen.

## 1.3 BGP-4 y la sesión entre vecinos

**BGP-4 (`RFC 4271`)** es el protocolo de enrutamiento **entre** sistemas autónomos. Corre **directamente sobre TCP**, en el **puerto 179**.

```
¿Por qué TCP y no un protocolo como UDP o como el de OSPF?

┌──────────────────────────────┬──────────────────────────────┐
│  Lo que BGP NECESITA         │  Lo que TCP LE DA            │
├──────────────────────────────┼──────────────────────────────┤
│  que los mensajes lleguen    │  fiabilidad (retransmisión)  │
│  y en orden                  │                              │
│  que no se dupliquen ni      │  orden (números de           │
│  se pierdan fragmentos       │  secuencia TCP)              │
│  control de congestión       │  control de flujo +          │
│  (Internet es best effort,   │  control de congestión       │
│  pero BGP no debe sumar      │  (ventana)                   │
│  carga extra al backbone)    │                              │
│  NO hay flooding: cada       │  conexión punto a punto      │
│  vecino recibe solo lo       │  con un solo vecino          │
│  que le corresponde          │                              │
└──────────────────────────────┴──────────────────────────────┘
```

Esa última fila es la clave conceptual: **BGP no inunda nada**. El flooding de LSA es un mecanismo de capa 3 con TTL y multidifusión; BGP usa una **sesión punto a punto TCP por vecino**. El precio es que hay que configurar los vecinos a mano (o por algún protocolo de configuración), y el beneficio es que el control de congestión de TCP protege el backbone de la señalización.

**El modelo de datos:** BGP es un **vecino a vecino** (*neighbor-to-neighbor*), no un **nodo a nodo** (*node-to-node*).

```
La diferencia es brutal:

LINK-STATE (nodo a nodo):
   R1 inunda a R2, R3, R4...  →  TODOS quedan con la MISMA base.
   Después cada uno calcula lo mismo. El resultado es idéntico.

BGP (vecino a vecino):
   R1 habla con R2.  R2 habla con R3.  R3 habla con R1. ...
   Cada sesión es una relación PRIVADA entre dos routers.
   R1 puede estar informado de algo que R4 no sabe, y al revés.
   ⇒ NO hay una base común, y por eso cada router decide con SU
     propia política. Esa es la razón de que BGP pueda tener
     "decisiones contradictorias" sin que nada esté roto.
```

> **Lo que esto implica para el forwarding:** las dos mejores rutas de BGP pueden **disputar el tráfico**. Si los routers vecinos toman cada uno un camino distinto para el mismo prefijo, el resultado es un **laqueo** (*load-balancing*) o un **tráfico en la dirección equivocada**. BGP resuelve eso en §1.7 con el **full-mesh iBGP** y el **NEXT_HOP** inalterado.

## 1.4 Path-vector: la diferencia clave con distance-vector

Un protocolo **path-vector** no manda "a la red X estoy a distancia 4". Manda **el camino completo**.

### AS_PATH: la lista, no el número

```
Lo que BGP anuncia:

  NLRI:      192.0.2.0/24          (el prefijo)
  AS_PATH:   20070  15169  20115   (por dónde se llegó)

... y el router receptor le PREPEND a su propio ASN cuando
re-anuncia la ruta:

  AS_PATH:   20115  20070  15169  20115     ← ¡no!  se rechaza. Ver abajo.
```

La regla que define a BGP es una sola:

```
┌──────────────────────────────────────────────────────────┐
│  Un router NO ACEPTA ni REEVALUA una ruta cuyo AS_PATH   │
│  contenga su PROPIO ASN.                                 │
│                                                          │
│  "Si mi ASN ya está en el camino que me ofrecen, alguien │
│   me está devolviendo una ruta que yo mismo anuncié.     │
│   Eso solo puede significar un bucle, así que la descarto." │
└──────────────────────────────────────────────────────────┘
```

Veamos el mecanismo paso a paso, que es lo que hay que saber dibujar:

```
t0   AS 20070 anuncia a AS 15169:   AS_PATH = [20070]
t1   AS 15169 anuncia a AS 20115:   AS_PATH = [15169, 20070]
t2   AS 20115 anuncia a AS 33000:   AS_PATH = [20115, 15169, 20070]
t3   AS 33000 anuncia a AS 20070:   AS_PATH = [33000, 20115, 15169, 20070]
     ┌──────────────────────────────────────────────────────┐
     │  20070 RECHAZA la ruta: su propio ASN (20070) ya está │
     │  en el AS_PATH. No la acepta, no la instala, no la    │
     │  anuncia. Fin. NO se genera ningún bucle.             │
     └──────────────────────────────────────────────────────┘

t4   20115 EXPIRA y avisa a 15169:   "dejá de usar la ruta por mí"
     15169 la borra y avisa a 33000, que la borra, que avisa a
     20070 (que ya nunca la tenía). Todo el camino se deshace
     solo, SIN conteo a infinito y SIN split horizon.
```

### Por qué es **exacto** y no probabilístico

Esta es la diferencia de fondo, y es la que más se pregunta.

| | **Distance-vector** (clase 32) | **Path-vector** (BGP) |
|---|---|---|
| Qué viaja | **Una estimación de distancia**, que puede estar vieja | **La historia exacta** del camino, que no puede envejecer |
| Cómo se detecta un bucle | **Contando**: si el costo sube, "algo anda mal". Con *hold-down* y *poison reverse*, es **probabilístico**: funciona en la mayoría de los casos y falla en algunos | **Mirando**: si mi ASN está en la lista, es un bucle. **Exacto, siempre** |
| Qué pasa con el rumor incorrecto | Se **realimenta**: `R2` le dice a `R1` que llega a destino a 3 usando a `R1`, y los dos suben la cuenta | No hay rumor de rutas, así que no hay nada que realimentar |
| Split horizon | **Imprescindible** | **Innecesario**, y no porque sea un parche que se olvidó |
| Convergencia con un error | Puede quedar oscilando entre dos valores para siempre | Converge al camino **más corto en número de AS** que la política permita |
| Límite de hops | RIP: 15, y el 16 es "infinito" | **No hay límite.** El AS_PATH crece, pero no hay tope duro |

```
¿Por qué BGP no cuenta a infinito? Porque no hay información
derivada que realimentarse.

El AS_PATH es un REGISTRO de lo que realmente pasó. Cuando AS
20070 recibe una ruta que dice "pasé por 20070", no está leyendo
un rumor UpdatedByR3: está leyendo un hecho físico que él mismo
genero. La información es verificable por construcción.
```

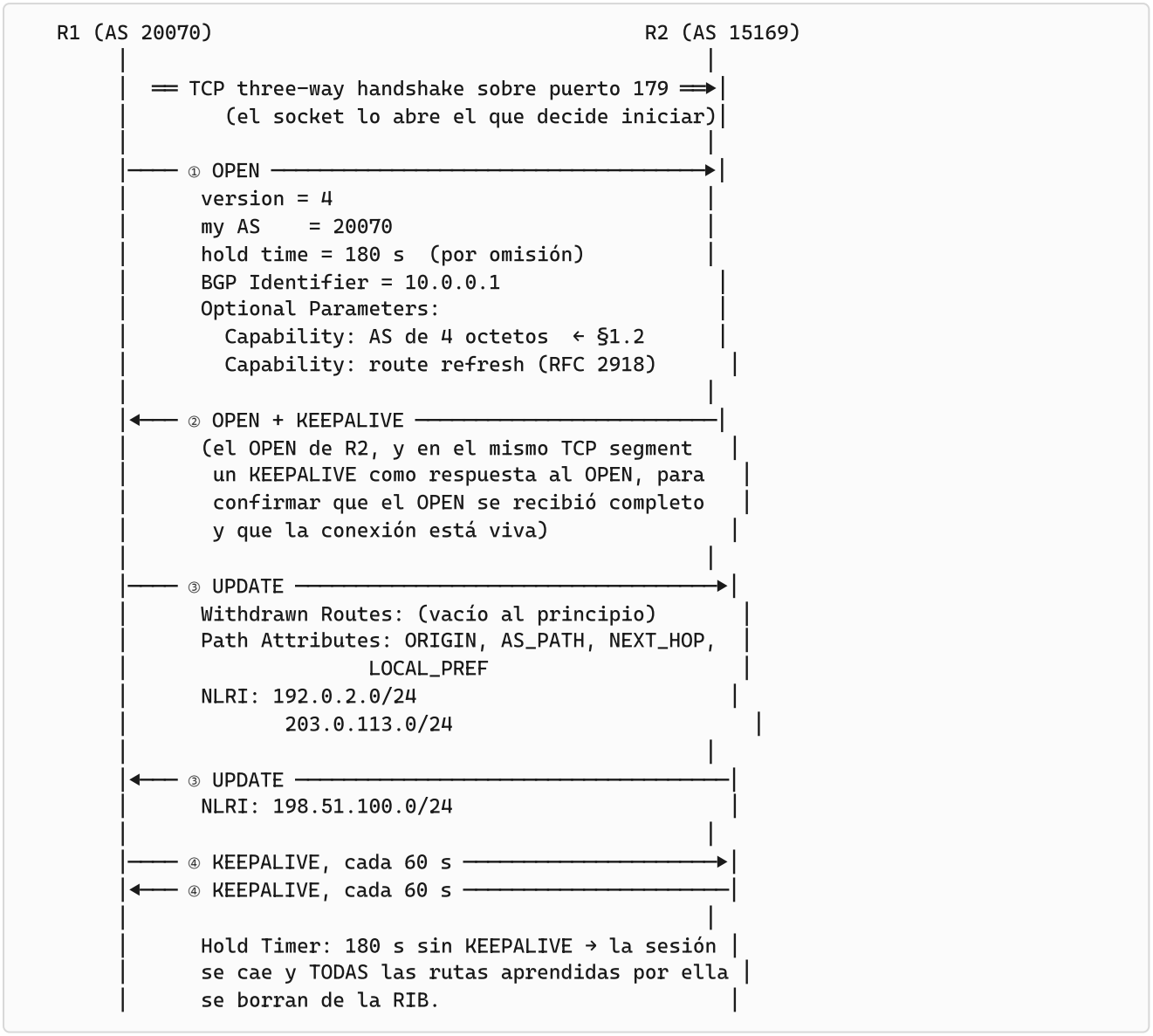
**Y el corolario incómodo, que hay que decir:** BGP es exacto contra bucles, pero **no es exacto contra errores de configuración**. Si `AS 20070` anuncia por error el prefijo de otro, ese error se propaga **a toda la Internet** en cuestión de segundos, sin que ningún mecanismo de conteo a infinito lo frene. La diferencia es que un bucle se autodetiene; **una mentira no**.

## 1.5 El handshake: OPEN → OPEN/ACK → UPDATE → KEEPALIVE

```
Los cuatro tipos de mensaje de BGP (header de 19 bytes: marker,
length, type):

1 = OPEN        2 = UPDATE      3 = NOTIFICATION
4 = KEEPALIVE   5 = ROUTE-REFRESH
```

## El diagrama del establecimiento




> ⚠️ **No inventar el número de RFC.** El *capability* de **route refresh** está definido por `RFC 2918`; ese número **no está** en nuestra lista de referencia, así que en el examen conviene nombrarlo sin número, o verificarlo antes. Todo lo demás que aparece acá (`OPEN`, `UPDATE`, `KEEPALIVE`, el hold time de 180 s y el keepalive de 60 s) está en `RFC 4271`.

### La máquina de estados

```
Idle
  │  (evento manual: se quiere hablar con este vecino)
  │  se abre el socket TCP al puerto 179
  ▼
Connect
  │  (se espera a que el TCP se complete)
  │  si falla, se reintenta cada ConnectRetryTime (120 s)
  ▼
Active
  │  (el OPEN fue enviado; se espera el OPEN del vecino)
  │  ⚠ En eBGP, si el OPEN no llega, se vuelve a Connect:
  │    el que no tiene el socket pasivo espera al que sí lo tiene.
  ▼
OpenSent
  │  (se recibió el OPEN: se comparan AS, version, hold time,
  │   Router ID. Si no coinciden → NOTIFICATION y Idle)
  │  se manda el KEEPALIVE de respuesta
  ▼
OpenConfirm
  │  (se espera el KEEPALIVE del vecino)
  ▼
Established   ← ACÁ se procesan los UPDATE y se anuncian rutas
  │
  └── si expira el Hold Timer, o llega un NOTIFICATION,
      o se cierra el TCP  →  Idle
```

> **El detalle que se pregunta:** el estado `Active` existe porque en **eBGP** los dos extremos no siempre pueden abrir la conexión al mismo tiempo (el "passive" no acepta). El que no está en *passive* espera; el otro lo busca. Sin ese estado, un router nunca se conectaría a uno que espera.

## 1.6 Los atributos de decisión y la tabla de desempates

BGP recibe **muchas** rutas al mismo prefijo y tiene que elegir una. Lo hace con un **proceso de decisión** determinista que evalúa atributos en un orden fijo.

## Los atributos, uno por uno

| Atributo | Significado | Dirección del criterio | Dónde se usa |
|---|---|---|---|
| **Weight** (no estándar) | Preferencia **local** del router | **Mayor** gana | Solo iBGP, y es propiedad del *vendor* |
| **LOCAL_PREF** | Preferencia que el AS le da a una ruta | **Mayor** gana | **Solo iBGP**. El valor por omisión es 100 |
| **Origen local** | ¿La ruta nació en este AS? | **Local gana** sobre todo lo demás | El atajo para no elegir un camino que vuelva a uno mismo |
| **AS_PATH length** | Cantidad de AS por los que pasó | **Menor** gana | eBGP e iBGP. **Es el criterio clásico de "corto"** |
| **Origin** | Cómo entró la ruta en BGP | `IGP` < `EGP` < `Incomplete` (en ese orden de preferencia) | Se usa si todo lo anterior empata |
| **MED** (*Multi-Exit Disc*) | Métrica del AS **vecino** | **Menor** gana | **Solo comparable entre rutas del mismo AS vecino** |
| **eBGP sobre iBGP** | Origen de la sesión | **eBGP gana** | Se usa si todo lo anterior empata |
| **Métrica IGP al NEXT_HOP** | Costo interno hasta el siguiente salto | **Menor** gana | El "hot potato" de §1.8 |
| **Router ID más bajo** | Desempate final | **Menor** gana | Solo para que sea **determinista** |

## La tabla de desempates, en orden

Esto es literal de `RFC 4271` más las extensiones, y es lo que hay que recite:

```
Bajar por la lista y quedarse con la que gana en el primer
criterio donde haya diferencia. Todas las rutas que ganan los
criterios anteriores siguen compitiendo.

1. Weight                     mayor gana       (local, no estándar)
2. LOCAL_PREF                 mayor gana       (solo iBGP)
3. Ruta originada localmente  local gana
4. AS_PATH más corto          menor gana       ★ el criterio clásico
5. ORIGIN                     IGP > EGP > Incomplete
6. MED                        menor gana       ★ solo entre el mismo AS vecino
7. eBGP sobre iBGP            eBGP gana
8. Métrica IGP al NEXT_HOP    menor gana       ★ el hot potato
9. Router ID más bajo         menor gana       (determinismo)
```

## NEXT_HOP y ORIGIN, en detalle

```
NEXT_HOP: la dirección IP a la que hay que enviar el paquete para
seguir por esta ruta.

· En eBGP, el que anuncia pone SU PROPIA dirección en el NEXT_HOP.
  "Si querés llegar por acá, entrá por mí."
· En iBGP, el que anuncia NO cambia el NEXT_HOP. Deja el que
  aprendió del eBGP de entrada.
  ⇒ de ahí sale la necesidad del **NEXT_HOP self** (§1.7).

ORIGIN: dice cómo se originó la ruta.
  IGP         → la ruta entró por un IGP (OSPF, IS-IS) [la mejor]
  EGP         → la ruta entró por un EGP legado, hoy es rarísimo
  Incomplete  → la ruta se redistribuyó desde otro protocolo, o el
                AS no lo originatorió: es el peor de los tres.
```

## Communities ( RFC 1997 )

Un atributo de 32 bits que lleva **etiquetas** que los routers interpretan con reglas. Es la forma de decir "trato especial" sin tocar la configuración de cada router:

| Community | Significado |
|---|---|
| `NO_EXPORT` | No la anuncies **fuera** de este AS |
| `NO_ADVERTISE` | No la anuncies **a nadie** |
| `NO_EXPORT_SUBCONFED` | No la anuncies fuera de la confederación |
| `65000:100` (formato `AS:value`) | **Definida por el operador**, con el AS como espacio de nombres |

```
Ejemplo real: un AS de tránsito marca con NO_EXPORT las rutas de
sus clientes directos. El resultado es que ese cliente es
alcanzable por los peers de ese AS, pero que los peers NO lo
anuncian hacia afuera. Es una forma de aplicar "hot potato"
invertido sin tocar una sola métrica.
```

**Y `AS_PATH prepending`**, que es lo que hace el 95 % de lasYPE policy en la práctica y que no es un atributo sino un **truco**:

```
Un AS que quiere salir menos por un vecino anuncia la misma ruta
VARIAS VECES con su propio ASN repetido al principio:

   AS_PATH real:       20070  15169  20115            (3 saltos)
   AS_PATH prepended:  20070  20070  20070  15169  20115   (5 saltos)

Los vecinos que usan "menor AS_PATH gana" lo evitan. No obliga a
nadie: un vecino que prefiera otra cosa (menor MED, mayor
LOCAL_PREF, o que tenga un peering con vos) lo ignora.
```

## 1.7 eBGP, iBGP, full-mesh y Route Reflection

### eBGP contra iBGP

```
eBGP:  entre routers de AS DISTINTOS
       ├─ el que anuncia REESCRIBE el NEXT_HOP con su propia IP
       ├─ el AS_PATH se le PREPENDea con su ASN
       ├─ el TTL del paquete BGP pasa a 1 (no sale del AS)
       └─ el LOCAL_PREF y el WEIGHT no se anuncian

iBGP:  entre routers del MISMO AS
       ├─ el que anuncia NO toca el NEXT_HOP
       ├─ el AS_PATH NO cambia (ya incluye el ASN propio)
       ├─ sí puede anunciar LOCAL_PREF
       └─ ⚠ REGLA CRÍTICA: una ruta aprendida de un iBGP NO se
          puede re-anunciar a OTRO iBGP
```

### La regla y por qué existe el full-mesh

```
La regla: "lo que aprendí de un vecino iBGP no se lo cuento a otro
vecino iBGP". Sin esta regla, un router que aprendió una ruta de
R1 se la pasa a R3, que se la pasa a R5, que se la pasa a R2, que
se la pasa a R1...  BUCLE. Y como el NEXT_HOP no se reescribe, los
paquetes se mandan en círculos.

Con la regla, ¿qué pasa?
R2 aprende de R1.
R3 NO puede aprender de R2.  ✗  ⇒ R2 y R3 no coinciden.

La solución clásica: que R1 se lo cuente a TODOS los demás.

       R1 ──────────► R2
       │ ╲            │
       │   ╲          │
       │     ╲        │
       │       ╲      │
       ▼         ╲    ▼
       R3 ──────────► R4

El FULL-MESH: todos con todos.
```

**El costo, que es la razón por la que existe Route Reflection:**

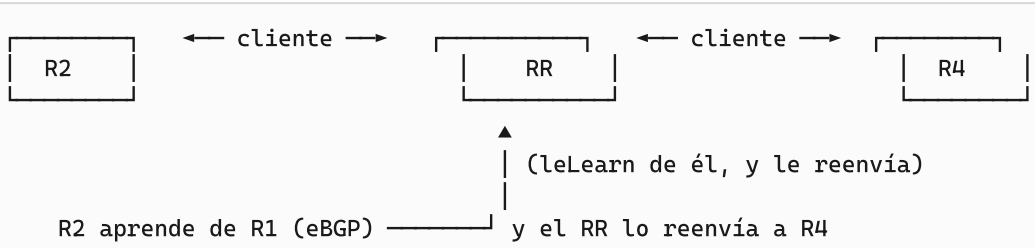
```
Sesiones iBGP necesarias =  N(N-1)/2

N = 5    →  10 sesiones
N = 10   →  45 sesiones
N = 20   → 190 sesiones
N = 50   → 1225 sesiones
N = 100  → 4950 sesiones
N = 200  → 19.900 sesiones

Con Route Reflection:  O(N)
```

## Route Reflection ( `RFC 4456` )

La idea: un router puede hacer de **espejo** de otro, y entonces sí puede pasarle rutas.




**Los dos atributos que hacen que el diseño sea seguro** — y son 2 preguntas de examen ensured:

| Atributo | Para qué |
|---|---|
| **ORIGINATOR_ID** | Identifica al router del cluster que **originó** la ruta. Si un cliente recibe por reflection una ruta cuyo `ORIGINATOR_ID` es él mismo, **la descarta**. Es el split horizon del path-vector |
| **CLUSTER_LIST** | Lista de los `CLUSTER_ID` s por los que pasó la ruta. Si vuelve a aparecer uno, **la descarta**. Evita bucles entre route reflectors |

```
Las dos reglas del protocolo:

1. Una ruta aprendida de un cliente iBGP NO se reenvía a los
   OTROS clientes (sí a los demás peers y al resto del cluster).
   → es la regla original, adaptada

2. Una ruta aprendida de un peer iBGP o de un cliente NO se
   reenvía a ningún otro cliente.

Y las dos reglas de NO reenvío:
· si mi ORIGINATOR_ID == el de la ruta → no la reenvío
· si mi CLUSTER_ID ya está en el CLUSTER_LIST → no la reenvío
```

**Y el detalle de implementación que se pregunta siempre:** con route reflection, el **NEXT_HOP sigue siendo el del eBGP de entrada**. El router interior que recibe la ruta tiene que **resolver ese NEXT_HOP** usando su IGP, o cambiarlo a sí mismo con **NEXT_HOP self** en su `UPDATE` . Sin eso, el router no tiene por dónde mandar el paquete.

## 1.8 "Hot potato": por qué un AS prefiere salir por el vecino más cercano

```
El caso clásico, que hay que dibujar:

┌──────────────── AS 20070 ─────────────────┐
│                                           │
│   [servidor local]                        │
│        │                                  │
│        ├──────────── enlace a AS 15169 ──────────►  destino
│        │      10 Mb/s   ·   1 hop         │    (en Internet)
│        │                                  │
│        └──────────── enlace a AS 20115 ──────────►  destino
│               1 Mb/s    ·   3 hops        │
└───────────────────────────────────────────┘

El servidor quiere alcanzar un destino que ambos vecinos
anuncian. Por cuál sale el router de borde?

La respuesta "obvia": por 15169, que a 10 Mb/s y a 1 hop.

La respuesta de BGP: por 20115, el que está MÁS LEJOS.
```

**¿Por qué?** Porque en la **última milla** no hay ancho de banda gratis. La conexión de **10 Mb/s** de `20070` a `15169` es un recurso que el operador tiene que pagar y mantener **entero**. Todo el tráfico que salga por ese enlace, aunque el cliente se vaya enseguida, consume ancho de banda propio y su capacidad de subida.

```
La analogía: un Starbucks en la esquina de tu casa y otraStarbucks
en la esquina de al lado de tu trabajo. El café de la esquina de tu
casa es más cómodo. El de tu trabajo es más barato para la empresa
porque ya está pagado de antemano por el alquiler de la oficina.

"hot potato" = "pasame la papa caliente":
sacala de tu casa y pasámela a quien está más lejos,
mientras la tuya se enfría en el escritorio.
```

### Cómo lo corrige el MED

`MED` (*Multi-Exit Disc*) es el atributo que le dice al AS vecino: **"salí por mí, y te paso el costo de salir por mí"**.

```
El proveedor 15169 anuncia a 20070 dos veces la misma ruta:

  Anuncio 1:  AS_PATH = [15169, 20115]        MED = 50
  Anuncio 2:  AS_PATH = [15169, 33000]       MED = 200

Con MED = 50, 20070 tiene un incentivo de 50 para salir por 15169,
que es la puerta de salida más barata para él.
Con MED = 200, el vecindario prefiere salir por la otra parte.
```

**Las dos trampas del MED**, que son pregunta clásica:

```
1. COMPARABLE SOLO DENTRO DEL MISMO AS VECINO.
   El RFC dice: comparar MED solo entre rutas que vinieron del
   MISMO neighbour AS. Si no, un vecino con MED = 0 (porque en
   SU red es normal) te va a ganar todas las comparaciones y te
   va a arrastrar a un camino objetivamente peor.

   Por eso todos los proveedores grandes tienen una
   CONFIGURACIÓN: "always compare med" / "never compare med" /
   "compare only if from the same AS". Es política, no estándar.

2. NO ES TRANSITIVO NI ORDENABLE.
   El MED compara ORIGENES (el vecindario), no la red de destino.
   Que AS-A tenga MED 50 hacia destino X y AS-B tenga MED 80
   hacia destino X no dice NADA sobre cuál tiene un camino más
   corto hasta Y. El MED es un número sin unidad: no se suman,
   no se promedian, no se propagan hacia atrás.
```

**Y el detalle que le da la vuelta a la aguja:** el "hot potato" se puede invertir **desde el lado del vecino** con `LOCAL_PREF`. Un AS puede subirle el `LOCAL_PREF` a las rutas que le llegan por un vecino determinado para que **todo su tráfico de salida** se vaya por ahí, sin tocar la configuración del vecino. Por eso `LOCAL_PREF` es el atributo **más usado** en la práctica y `MED` es el segundo.

## 1.9 Por qué BGP no cuenta a infinito ni necesita split horizon

Recapitulamos, porque es el cierre conceptual de las clases 32, 33 y 34:

```
DISTANCE-VECTOR                    LINK-STATE              PATH-VECTOR
───────────────                    ──────────              ───────────
viaja: una ESTIMACIÓN              viaja: un HECHO         viaja: un HECHO
       de distancia                       local                   de historia

¿se puede realimentar?  SÍ         ¿se puede realimentar?  NO
¿qué lo detecta?        CONTAR      ¿qué lo detecta?        NADA: el árbol
¿cuán exacto es?        PROBABILÍSTICO ¿qué lo detecta?     NO HAY CAMINO
                         (split horizon,                      QUE VOLVER
                         hold-down)                           (es un árbol)

rip → split horizon → pero igual      ospf → flooded →      bgp → AS_PATH →
puede oscilar                           inundación O(N)        rechazo exacto
```

**El argumento completo, en tres pasos, que es lo que hay que escribir:**

1. **No hay estimación que realimentarse.** En DV, el problema es que `R2` comunica a `R1` una distancia **calculada**, que puede estar vieja y que incluye a `R1` en el camino. En BGP, lo que viaja es el **AS_PATH**, que es un **registro verificable** de los AS por los que pasó el anuncio. No hay nada derivado.
2. **El lazo se reconoce por inspección.** Cuando `AS 20070` recibe un AS_PATH que contiene `20070`, el ciclo **está escrito en el mensaje**. No hace falta detectarlo contando: se lee.
3. **La detección es estructural, no estadística.** El *split horizon* de DV es una **heurística**: "no le digas a nadie lo que te llega por él" funciona en el 99 % de los casos y falla en algunos. El filtro de `AS_PATH` funciona en el **100 %**, porque la condición que chequea es **necesaria y suficiente** para que exista un lazo. No es "una buena heurística contra bucles": es **la definición** del bucle.

```
Y hay una diferencia más, que es la que más se olvida:

BGP NO TIENE CONVERGENCIA EN EL SENTIDO DE OSPF.

En OSPF, cuando el flooding termina, TODOS los routers tienen el
MISMO árbol. En BGP, cuando "terminó", cada router tiene SU propio
mejor camino según SU política, y puede ser distinto del de su
vecino. Eso no es un bug: es el diseño.

El resultado es que BGP no tiene una noción de "estado estable
global", y por eso el tráfico puede oscilar entre dos rutas
válidos durante minutos (y a veces, con prefijos anunciados y
retirados rapidísimo, durante días).
```

## 1.10 Seguridad: por qué BGP no es seguro

BGP corre sobre **TCP/179**, y **TCP no cifra ni autentica nada**. Todo lo que viaja entre dos routers BGP es **texto claro y sin firma**.

```
┌──────────────────────────────────────────────────────────┐
│  LO QUE TCP TE DA          │  LO QUE BGP NECESITA        │
│  · fiabilidad              │  · que el vecino SEA        │
│  · orden                   │    quien dice ser           │
│  · control de flujo        │  · que el CAMINO que dice   │
│                            │    sea el que realmente     │
│                            │    recorrió                 │
│                            │  · que NADIE más haya       │
│                            │    interceptado el mensaje  │
└──────────────────────────────────────────────────────────┘
                ↑ ↑ ↑
    ninguna de las tres cosas la da TCP
```

**Los dos ataques[l] que eso habilita**, y que hay que nombrar:

| Ataque | Qué hace |
|---|---|
| **Hijacking de sesión / MITM** | Alguien en el camino intercepta el TCP y manda mensajes BGP falsos. Como no hay cifrado, puede **injectar rutas** y **redirigir tráfico** |
| **Route leak** | Un AS **legítimo y bien configurado** anuncia prefijos que no le pertenecen. No hay ataque: es un error (o una ambición desmedida). Es **el fallo más común de BGP en la práctica** |

### El mecanismo clásico: MD5 de sesión ( `RFC 2385` )

La primera respuesta fue **firmar cada mensaje**:

```
· Se calcula un MD5 sobre (password + mensaje BGP) y se lo manda.
· El vecino recalcula y compara.
· Si no coincide, el mensaje se descarta.
· Se protege contra el MITM. NO protege contra nadie que esté
  legítimamente en el camino y tenga la clave.
· Es un mecanismo SIMÉTRICO: ambas partes tienen la misma clave.
```

Es `RFC 2385` (2000), y fue **el** mecanismo de BGP security durante dos décadas.

### La respuesta moderna: los roles BGP (`RFC 9234`, 2022)

```
La idea: en lugar de PROBAR quién es el vecino (con criptografía),
DECLARAR qué relación comercial se supone que hay entre los dos,
y rechazar cualquier mensaje que la contradiga.

┌──────────────────────────────────────────────────────────┐
│  ROLES (declarados en el OPEN y en cada UPDATE)          │
│                                                          │
│  Provider  → "soy tu proveedor (tú me pagás)"            │
│  Customer  → "soy tu cliente (me pagás vos)"             │
│  Peer      → "somos pares, ninguno le paga al otro"      │
│  RS        → "Route Server"                              │
│  RS-Client → " soy cliente de un Route Server"           │
│  Lateral   → relación interna de confianza               │
│  Internal  → uso interno del AS                          │
└──────────────────────────────────────────────────────────┘

EJEMPLO, que es el que hay que saber:

ANTES (solo MD5):
   AS A es cliente de AS B. B anuncia a A un prefijo que en
   realidad es de C, que es cliente de A.
   Con MD5, A no tiene forma de saberlo: el mensaje es
   criptográficamente válido. A lo acepta y lo anuncia. LEAK.

DESPUÉS (roles):
   B anuncia a A el prefijo de C con ROL = Provider.
   A mira su configuración: "B es mi proveedor", y los clientes de
   A (como C) NUNCA se anuncian por un proveedor. CONTRADICCIÓN.
   A rechaza la ruta.

⇒ El rol es una AFIRMACIÓN VERIFICABLE CONTRA LA CONFIGURACIÓN
  LOCAL. Y la configuración local es lo único que no se puede
  falsear con un ataque en el camino.
```

`RFC 9234` es de **mayo de 2022**. Su texto dice explícitamente que reemplaza al `RFC 2385` como mecanismo de referencia: hoy MD5 es **legado**.

### RPKI / ROA: validar el origen del prefijo

MD5 y los roles protegen **el camino**. RPKI protege **la cosa misma**:

```
El problema que MD5 y los roles NO resuelven:

AS A (legítimo, bien configurado, sin bugs) anuncia el prefijo
203.0.113.0/24, que NO le pertenece. Legalmente puede hacerlo:
nadie se lo impide. Y lo anuncia a todo el mundo.

¿Cómo sabemos que no le pertenece? Porque hay un REGISTRO.
```

| Pieza | Qué es |
|---|---|
| **ROA** (*Resource Origin Authorization*) | Un documento firmado digitalmente que dice: **"el AS X está autorizado a originar el prefijo P"**. Cada RIR lo publica |
| **RPKI** (*Resource PKI Infrastructure*) | La **infraestructura de claves públicas** que firma los ROA. Equivale a la PKI del DNI, pero para prefijos |
| **RTR** (*RPKI to Router*) | El **protocolo** que le lleva los ROA al router, para que pueda validarlos. `RFC 6810` |
| **Validación** | El router recibe el ROA de un prefijo y de su AS de origen, y lo compara con la ruta que le llega. `RFC 6811` |

```
Los tres resultados posibles de la validación de una ruta:

VALID       el prefijo tiene un ROA y el AS de origen coincide
INVALID     el prefijo tiene un ROA y el AS de origen NO coincide
            → el AS está anunciando algo que no le pertenece. Es un
              ataque o un error. → DESCARTA.

NOT FOUND   el prefijo no tiene ROA. No dice nada. → "no sé", y
            BGP sigue con su decisión de todas formas.

⚠ Y ACÁ ESTÁ LA TRAMPA que hay que aclarar siempre: si el 99 % de
los prefijos de Internet están "Not Found", RPKI no cambia
ABSOLUTAMENTE NADA para ese 99 %. Solo sirve donde hay ROA. Es
por eso que la despliegue real es bajo: no falla, pero tampoco
hace casi nada todavía.
```

**El punto clave para el examen:** RPKI **no cifra ni autentica la sesión**. Valida el **origen declarado del prefijo**, leyendo una cadena de confianza externa a BGP. Es **ortogonal** a MD5 y a los roles: los tres resuelven cosas distintas y ninguno reemplaza a los otros.

## ⚠️ Actualización a 2026

**Sigue igual — de memoria, sin dudar:** - **BGP-4 (`RFC 4271`)** sigue siendo la especificación vigente. El "4" es la versión, y desde 1994 no cambió el formato. - **`AS_PATH` es un *path-vector***: la lista de AS por los que pasa la ruta, y **un AS no acepta una ruta cuyo AS_PATH contenga su propio AS**. Eso es lo que hace que BGP sea *path-vector* y no *distance-vector*. - **Sesión sobre TCP/179**: OPEN → OPEN/ACK → UPDATE → KEEPALIVE. Hold time 180 s, keepalive 60 s. - **eBGP contra iBGP**; la regla de "no re-anunciar a iBGP lo aprendido de un iBGP" obliga al **full-mesh iBGP**, y de ahí salen las **Route Reflection (`RFC 4456`)**. - **Atributos**: `LOCAL_PREF` (↑ mejor), longitud de `AS_PATH` (↓ mejor), `MED` (↓ mejor, **comparable solo entre rutas del mismo AS vecino**), `ORIGIN`, `NEXT_HOP`, **Communities (`RFC 1997`)**. - **"Hot potato"**: cada AS prefiere salir por el eBGP de **menor métrica IGP**. `MED` es la herramienta para anular esa preferencia. - **OSPFv2 (`RFC 2328`)** = link-state, SPF/Dijkstra, áreas, DR election, multicast 224.0.0.5/6, TTL 1 en el hello. - **RIPv2 (`RFC 2453`)**: métrica ≤ 15, poison reverse (16), split horizon, triggered updates, hold-down. **Sigue siendo *Internet Standard*, pero está deprecado de facto en Internet**: sobrevive en laboratorios y campus.

**Cambió:**

- `RFC 9234` **(may-2022): BGP sin MD5 de sesión.** Define **roles BGP** (Provider / Customer / Peer / Lateral / Internal) declarados en OPEN y UPDATE para prevenir y detectar *route leaks*. Reemplaza al

`RFC 2385` (MD5 de sesión) como mecanismo de referencia.

- **RPKI / ROA**: `RFC 6810` (RTR) y **`RFC 6811`** (BGP Prefix Origin Validation). Maduró de idea a **despliegue real**, pero con el matiz de cobertura de §1.10: sin ROA no hay validación. La seguridad de BGP pasó de "confiar en AS_PATH" a "**validar origen con RPKI**".
- **`BGP-LS`** → **`RFC 9552`** (dic-2023), obsoleta RFC 7752.
- **EVPN (`RFC 7432`) + VXLAN (`RFC 7348`)**: hoy son **el estándar de overlay de datacenter**, no solo académico. Y como el plano de control de EVPN es **BGP**, esto es lo que se conoce como **"BGP para datacenters"**.
- **Segment Routing**: `RFC 8402` (arquitectura, 2018), `RFC 8660` (SR-MPLS), **`RFC 8986`** (SRv6). Es el **sucesor de facto** de MPLS LDP/RSVP-TE en despliegues nuevos: *source routing* con SIDs y **estado solo en el borde de ingreso**. SRv6 codifica los SIDs en la cabecera IPv6 y **elimina el MPLS por completo**.
- **OSPFv3 (`RFC 5340`)**, IS-IS para IPv6 (`RFC 5308`): mecanismos idénticos, independientes de IPv6.
- **AS de 32 bits**: `RFC 4893` + `RFC 6793`; notación decimal `RFC 5396`. **Ya no existe el problema del AS de 16 bits.**
- **BMP (`RFC 7854`)** para monitorear BGP desde fuera.

**Consecuencia para el examen:** si el enunciado dice "describa BGP" o "explique el path-vector", la respuesta con `AS_PATH` + rechazo del propio ASN + "no cuenta a infinito porque no viaja ninguna estimación" es la que suma. Si menciona seguridad, sumá que **MD5 es legado** y que hoy el mecanismo de referencia son los **roles BGP de `RFC 9234`**, con **RPKI** validando el origen del prefijo.

## 1.11 La tabla resumen: link-state, distance-vector y path-vector

| | **Distance-vector** (RIP) | **Link-state** (OSPF, IS-IS) | **Path-vector** (BGP) |
|---|---|---|---|
| **Qué viaja** | Vector de distancias | Un **LSA** (hecho local) | El **AS_PATH** (historia del camino) |
| **Qué calcula** | Bellman-Ford (iterativo) | Dijkstra (SPF) | Proceso de decisión por atributos |
| **Alcance** | Un AS | Un área / un IGP | **Internet** |
| **Topología** | No la conoce | **Idéntica en todos** del área | Cada vecino ve una **vista distinta** |
| **¿Hay bucle?** | **Sí**, conteo a infinito | No (árbol) | No (rechazo por `AS_PATH`) |
| **¿Cómo se rompe?** | Split horizon, hold-down | Los tres controles del flooding | Exacta, no probabilística |
| **¿Quién decide?** | La distancia mínima | El costo de la métrica | **La política**, atributo por atributo |
| **Transporte** | UDP 520 | IP protocolo 89 | **TCP 179** |
| **Escala** | LAN / campus | Red corporativa, backbone | **Los ~70.000 AS** |
| **Ejemplo del curso** | `RFC 2453` | `RFC 2328` | `RFC 4271` |

# PARTE 2 — PRÁCTICA RESUELTA

## 2.1 Ejercicio 1 — La tabla de desempates aplicada a cuatro rutas

**Datos.** En el router `R7` del AS **20070** llegan cuatro rutas al prefijo `198.51.100.0/24`, todas aprendidas por **iBGP** de `R1`, `R3`, `R5` y `R6` respectivamente:

| # | Vecino que la trajo | `WEIGHT` | `LOCAL_PREF` | `AS_PATH` | `ORIGIN` | `MED` | Tipo de sesión |
|---|---|---|---|---|---|---|---|
| A | `R1` | 100 | 100 | `20115 15169 20070` | IGP | 50 | iBGP |
| B | `R3` | 100 | **200** | `33000 20115 20070` | IGP | 10 | iBGP |
| C | `R5` | **300** | 100 | `15169 20070` | Incomplete | 5 | iBGP |
| D | `R6` | 100 | 100 | `20115 15169 20070` | IGP | 50 | iBGP |

**Resolución paso a paso, aplicando el orden de §1.6.**

**Paso 1 — `WEIGHT` (mayor gana).** C tiene 300, los otros 100. **C queda primera; A, B y D siguen.**

*Interpretación:* el `Weight` es el primer corte y es **local**: no se anuncia a nadie. Configurarlo es la forma más rápida de forzar una salida.

**Paso 2 — `LOCAL_PREF` (mayor gana).** Solo compite entre las que pasaron el paso 1: B tiene 200, A y D tienen 100. **Gana B.**

*Interpretación:* `LOCAL_PREF` solo se usa en iBGP y es **la** herramienta de política interna. Acá dice: "la ruta que me trajo `R3` es la que yo prefiero", aunque el `AS_PATH` sea más largo.

**Paso 3 — `AS_PATH` más corto.** Se descarta C (perdió en el paso 1) y se comparan las que quedan: A y D tienen `20115 15169 20070` (**3 AS**), B tiene `33000 20115 20070` (**3 AS**). **Empate, todas siguen.**

*Interpretación:* ojo con esto. B tiene el `LOCAL_PREF` más alto pero **el mismo número de saltos**. Si tuviera 4, seguiría ganando: el `LOCAL_PREF` gana **antes** que la longitud del `AS_PATH`. Eso es exactamente lo que significa "el `LOCAL_PREF` es la política del AS, y el `AS_PATH` es solo un criterio de desempate".

**Paso 4 — `ORIGIN` (IGP > EGP > Incomplete).** A, B y D tienen IGP. **Empate, todas siguen.**

*Interpretación:* C tenía `Incomplete` porque la ruta fue redistribuida desde otro protocolo. Habría sido eliminada en este paso **si no hubiera perdido antes**. El orden importa: el `ORIGIN` se mira **después** del `AS_PATH`, no antes.

**Paso 5 — `MED` (menor gana).** A y D tienen 50, B tiene 10. **Gana B.**

**Paso 6 — eBGP sobre iBGP.** Las tres son iBGP. Empate.

**Paso 7 — Métrica IGP al `NEXT_HOP` (menor gana).** Supongamos que la ruta de B tiene `NEXT_HOP = 192.0.2.7`, que está a **8 saltos** por el IGP, mientras que las de A y D apuntan a `192.0.2.3`, a **3 saltos**.

**¡B gana el paso 7!** Y es acá donde aparece la paradoja:

```
B llegó en el paso 2 por tener LOCAL_PREF más alto.
B llega al final porque su NEXT_HOP está más lejos dentro del AS.

Si el operador NO hubiera configurado LOCAL_PREF en R3,
el resultado habría sido A (o D), que sale por el vecino de 3 hops.

O sea: el "hot potato" (§1.8) gana por DEFAULT, y los operadores
lo deshacen a propósito subiendo LOCAL_PREF para el vecino que
les conviene. Configurar LOCAL_PREF es, literalmente,
comprarle el tráfico a otro AS.
```

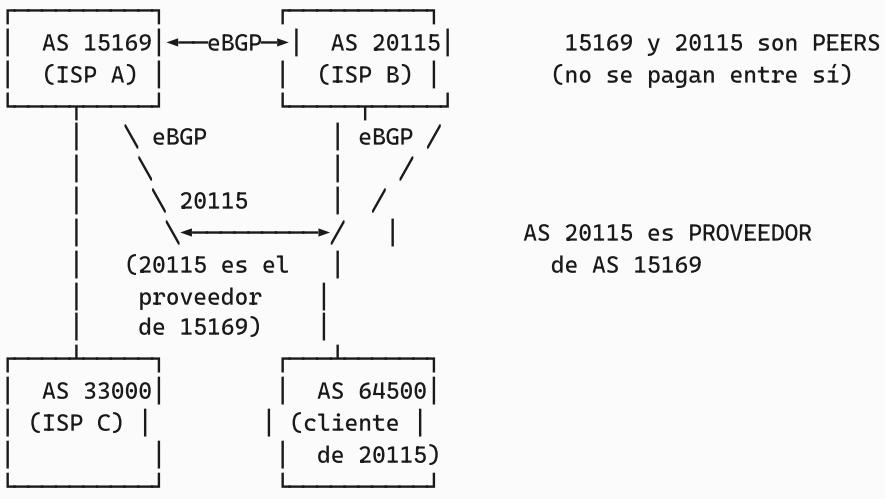
**Paso 8 — `Router ID` más bajo.** Si A y D siguen empatadas (mismo `MED`, mismo `NEXT_HOP`), **gana la que tenga el Router ID más bajo**. Existe solo para que el resultado sea **determinista**: dos routers con la misma configuración tienen que elegir lo mismo, o el tráfico se parte al azar entre dos caminos.

### ⚠️ Errores típicos

- **Comparar `MED` entre rutas de AS vecinos distintos.** No se puede (§1.8).
- **Confundir `LOCAL_PREF` con `AS_PATH length`.** El primero se mira **primero** y gana aunque el segundo sea peor.
- **Asumir que el `AS_PATH` más corto gana siempre.** No: solo después de `WEIGHT` y `LOCAL_PREF`.
- **Olvidar el paso 8.** Los desempates tienen que ser deterministas o los routers vecinos toman caminos distintos y se rompe el forwarding.

## 2.2 Ejercicio 2 — Topología de tres AS, paso a paso

**Datos.** Topología:




**a) ¿Cómo ve el prefijo de `AS 64500` cada router?**

El orden de propagación importa: primero `64500` se lo dice a `20115`; `20115` decide si lo anuncia a `15169` y a `33000`; y cada uno de esos lo anuncia o no a sus clientes.

| Quien anuncia | `AS_PATH` que genera | Nota |
|---|---|---|
| `64500` → `20115` | `64500` | eBGP: el que anuncia preprende su ASN |
| `20115` → `15169` | `20115 64500` | Decisión: es cliente, **lo anuncia** |
| `15169` → `33000` | `15169 20115 64500` | Decisión: tránsito, **lo anuncia** (si lo vende) |
| `20115` → `33000` | `20115 64500` | **También se lo anuncia directo** |

*Interpretación:* `33000` recibe **dos** rutas al mismo prefijo: una de largo 3 y otra de largo 2. Se queda con la de **2** ( `20115 64500` ), porque es su vecino directo y ya tiene el prefijo en su tabla. Y `15169` prefirió **no** anunciárselo a `33000` (decisión de política: `20115` no le vende tránsito a un competidor).

**b) El `AS_PATH` que recibe `33000` es `20115 64500` . ¿Y si `64500` se cae?**

```
64500 cae y abre una sesión con 33000 (multihoming).
Ahora 33000 tiene DOS proveedores para el mismo prefijo.

64500 → 20115 → 15169 → (vuelta a 33000):  RECHAZADO. 64500 ya está
en el AS_PATH. Exactamente el mecanismo de §1.4.

64500 → 33000 (directo):   AS_PATH = [64500]. Aceptada.
→ 33000 prefiere 1 salto sobre 4. Decide mal, pero es SU decisión.
```

**c) ¿Qué pasa si `20115` anuncia a `15169` el prefijo de `64500` con un `LOCAL_PREF` bajo?**

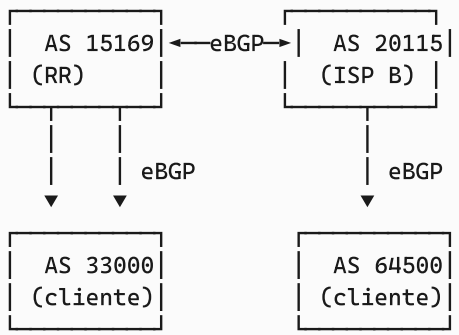
```
15169 recibe AS_PATH = [20115, 64500] con LOCAL_PREF = 150.
15169 ya tenía su propia ruta hacia ese prefijo? NO: 64500 es
cliente de 20115, no de 15169.

→ 15169 la acepta, la instala, y la anuncia a 33000 con
  LOCAL_PREF = 150 (propagando el valor).

OJO: 15169 NO le puede cambiar el AS_PATH. Sigue siendo
[15169, 20115, 64500]. Lo único que controla es a quién se lo
anuncia y con qué LOCAL_PREF.

Y acá está la diferencia con el AS_PATH: el AS_PATH describe el
CAMINO FÍSICO (no se puede mentir sin romper la detección de
bucles); el LOCAL_PREF describe la INTENCIÓN (que se puede mentir,
pero solo en tu propio AS: es una política interna).
```

**d) Sumá Route Reflection al mismo dibujo, con `15169` como reflector.**




```
15169 aprende el prefijo de 64500 de 20115 por eBGP.
15169 NO se lo anuncia a 64500 (es su cliente y no tiene sentido).
15169 se lo anuncia a 33000 como cliente suyo, por REFLECTION,
con AS_PATH = [15169, 20115, 64500] y ORIGINATOR_ID = 15169.

Ahora el agregado: en lugar de 3 sesiones iBGP (full-mesh)
hay 2. Con N routers en el cluster: N en lugar de N(N-1)/2.
```

## 2.3 Ejercicio 3 — Por qué BGP necesita full-mesh iBGP y qué rompe Route Reflection

**Enunciado.** En un AS con `R1, R2, R3, R4`, `R1` aprende una ruta de un vecino **eBGP**. Explicá por qué `R2` y `R3` no pueden learnla el uno del otro, qué problema crea eso, y qué rompe exactamente Route Reflection.

**a) Por qué la regla "no re-anuncio lo aprendido de un iBGP".**

```
R1 (eBGP)  ── ruta r ──►  R2
R1          ── ruta r ──►  R3
R1          ── ruta r ──►  R4

R2 y R3 son iBGP entre sí. Si R2 le dijera a R3 "ruta r":
   · R3 la aprende y la anuncia a R1 (que es su vecino iBGP)
   · R1 ya la tiene, pero no puede distinguir "la que yo anuncié"
     de "la que me devolvió R3"
   · el NEXT_HOP no cambió: sigue siendo el del eBGP de entrada
   · ⇒ R3 manda el paquete a un vecino que se lo devuelve a R3.
     BUCLE DE PAQUETES.

Y el AS_PATH no lo detiene, porque el bucle es entre routers
DEL MISMO AS: el AS_PATH es idéntico en los dos caminos.
Es un bucle de capa 3, no de capa 2.
```

**b) El problema que crea la regla.**

```
Con la regla, R2 NO puede obtener la ruta de R3, ni R3 de R2.
Para que todos tengan la MISMA tabla (que es lo que exige un
protocolo de enrutamiento: forwarding consistente), R1 tiene
que anunciársela a CADA uno.

Sesiones necesarias = N(N-1)/2, todos contra todos.

Consecuencias:
· N = 50  →  1225 sesiones TCP, cada una con su temporizador,
             su buffer y su CPU de keepalive.
· En un operador grande, N puede ser 200: 19.900 sesiones. Y cada
  router tiene que mantenerlas todas vivas.
· Cada caída de una sesión obliga a recalcular.
```

**c) Qué rompe Route Reflection, y la respuesta corta es "nada, pero con dos excepciones".**

```
Route Reflection agrega una excepción a la regla: el RR (Route
Reflector) SÍ puede pasarle a sus clientes una ruta que aprendió
de un peer o de otro cliente.

Eso no rompe nada siempre que se respeten las DOS reglas nuevas:

┌─ EXCEPCIÓN 1: ORIGINATOR_ID ──────────────────────────────────┐
│ Si una ruta llega a un cliente con ORIGINATOR_ID = MI PROPIO   │
│ Router ID, el cliente la DESCARTA.                            │
│                                                               │
│ Por qué: es el split horizon del cluster. Si R3 learns de R2  │
│ una ruta que R2 reflejó desde R1, y esa ruta dice             │
│ ORIGINATOR_ID = R3, es porque el camino pasó por R3. Aceptarla │
│ es aceptar un bucle.                                          │
└───────────────────────────────────────────────────────────────┘

┌─ EXCEPCIÓN 2: CLUSTER_LIST ───────────────────────────────────┐
│ Si mi CLUSTER_ID YA ESTÁ en el CLUSTER_LIST de la ruta, la    │
│ DESCARTO.                                                     │
│                                                               │
│ Por qué: si el CLUSTER_LIST vuelve a aparecer, la ruta pasó   │
│ por un route reflector del MISMO cluster. Entre dos reflectores │
│ del mismo cluster se usa una sesión iBGP normal (con la regla  │
│ original), así que eso es un bucle.                           │
│                                                               │
│ El CLUSTER_ID se configura. Lo normal es poner el Router ID del │
│ RR, para que un solo reflector sea un solo cluster. Si querés  │
│ que dos reflectores redundantes NO se confirmen entre sí, les ponés │
│ el mismo CLUSTER_ID.                                          │
└───────────────────────────────────────────────────────────────┘
```

**d) El problema que Route Reflection NO resuelve, y que es la pregunta fina.**

```
⚠ Route Reflection no arregla la CONSISTENCIA de la tabla.

Con full-mesh, TODOS los routers del AS tienen exactamente la
misma tabla (dentro de lo que la política permita).

Con route reflection, cada router tiene una tabla CORRECTA pero
DISTINTA de la de su vecino: le falta lo que seannounced por el
camino que no pasó por él.

Eso está bien mientras cada router use la ruta para DECIDIR. Pero
si un router usa su tabla para REANUNCIAR hacia afuera sin pasar
por un RR, rompe el modelo. Por eso los RR solo se usan DENTRO del
AS, y los routers de borde (BORDER routers) estántightmente
controlados.
```

---

# PARTE 3 — PROBLEMAS DE EXAMEN DE FING

## 3.1 ⭐ Examen 2001-08, Problema 2 — Implementar un algoritmo de ruteo jerárquico

> **Fuente:** Examen `eirc0108`, 20 de agosto de 2001 (2 páginas). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0108.pdf **Texto del examen:**
>
> > Implementar un algoritmo de ruteo jerárquico para una red de datagramas. Definir la estructura de las tablas de ruteo y los paquetes utilizados.
>
> ⚠ **Nota:** el PDF **no publica solución** de este problema (el examen viene sin soluciones). Lo que sigue es *(solución propia)*, y es central para esta clase porque **es literalmente lo que hace BGP**: particionar el espacio de direcciones en dominios administrativos y que cada dominio anuncie el resto como un **agregado**, en vez de que cada router conozca cada ruta.

### El enunciado, reencuadrado

El problema pide un algoritmo de **ruteo jerárquico**. Hay que justificar tres cosas:

1. **Qué es jerárquico** y por qué no alcanza con un grafo plano.
2. **La estructura de las tablas**.
3. **Los paquetes** del protocolo.

### a) Por qué jerárquico, y no plano

**Un grafo plano no escala.** Supongamos la red del problema: `N` routers, cada uno con una entrada por cada destino. La tabla tiene `N` filas, y **cambia completa cada vez que cambia un enlace**: `O(N)` actualizaciones, cada una de `O(N)` filas.

La idea de jerarquizar es agrupar los destinos en **grupos** y anunciar el grupo entero como una sola entrada:

```
SIN JERARQUÍA (plano):                    CON JERARQUÍA:
──────────────────────                    ──────────────────
tabla de R1:                              tabla de R1:
  red_A → 10.0.0.2                          10.0.0.0/8   → 10.0.0.2
  red_B → 10.0.0.2                          (un prefijo que
  red_C → 10.0.0.2                           cubre TODO)
  ...
  red_Z → 10.0.0.2
N entradas, N cambios por evento

El interior de 10.0.0.0/8 NO se anuncia hacia afuera:
los routers de la red "de arriba" (el AS vecino, el router de
borde) solo saben que "todo lo que empieza con 10. está detrás
de 10.0.0.2". No saben cuántas redes hay ni dónde.
```

**Las dos propiedades que hace aparecer la jerarquía**, y que son las que hay que nombrar:

1. **Agregación**: varios prefijos se announced como **un** superprefijo. El borde anuncia `10.0.0.0/8` y se olvida de los detalles.
2. **Ocultamiento de la topología interna**: los routers externos **no** conocen ni la topología ni el número de prefijos del dominio. Solo conocen **un punto de entrada**.

```
Y acá aparece la diferencia con el jerarquizado "de áreas" de OSPF,
que es la que más se confunde:

┌──────────────────────────┬──────────────────────────────────────┐
│  ÁREA de OSPF            │  JERARQUÍA DE BGP                    │
├──────────────────────────┼──────────────────────────────────────┤
│ se announce la           │ se announce un SUPERPREFIJO          │
│ INFORMACIÓN (cada LSA)   │ (la métrica y el resultado)          │
│ del interior             │                                      │
│                          │                                      │
│ los routers internos     │ los routers externos no saben        │
│ necesitan la información │ ni la topología ni el detalle        │
│ completa para calcular   │                                      │
│ el camino ÓPTIMO         │                                      │
│                          │                                      │
│ se pierde optimalidad    │ no hay "camino óptimo" que           │
│ DENTRO del área          │ perder: la agregación es correcta    │
│                          │ por definición                       │
│                          │                                      │
│ el estado es por ENLACE  │ lo que se announce es la RUTA        │
│ y hay que calcularlo     │ AGREGADA y su política               │
└──────────────────────────┴──────────────────────────────────────┘
```

> **Ésta es la respuesta conceptual más importante del problema:** OSPF jerarquiza **para escalar el cálculo** y BGP jerarquiza **para escalar el announcement**. Los dos resuelven el mismo problema (la tabla plana no escala) con herramientas opuestas: uno publica los hechos y esconde el camino, el otro publica el camino y esconde los hechos.

### b) Estructura de las tablas de ruteo

Como no hay texto de apoyo en el enunciado, **hago las suposiciones y las justifico**, que es lo que el propio examen pide en otros problemas.

**Suposición 1: la jerarquía se define por un prefijo de la dirección IP.** Es lo coherente con el modelo TCP/IP del curso, y evita tener que inventar un esquema de nombres nuevo.

**Suposición 2: cada router tiene tres tablas**, porque en un protocolo jerárquico hay tres preguntas distintas:

| Tabla | Qué contiene | Tamaño |
|---|---|---|
| **T_int** (intradominio) | Las rutas **exactas** de todos los prefijos del dominio propio | Grande: $O(n)$ |
| **T_agg** (agregados) | Los superprefijos que este router **anuncia hacia afuera** | Chica: $O(g)$, con $g \ll n$ |
| **T_ext** (externos) | Los superprefijos de los dominios vecinos | Chica: $O(d)$ |

```
┌──────────────────────────────────────────────────────┐
│  Tabla T_AGG, formato de una entrada:                │
│                                                      │
│   prefijo_agregado   →  siguiente_salto  (interfaz)  │
│   10.0.0.0/8          →  10.0.0.2        eth0        │
│   172.16.0.0/12      →  10.0.0.3        eth1        │
│   192.168.0.0/16     →  10.0.0.4        eth2        │
│                                                      │
│  Y ENCIMA, una entrada que ES el comportamiento de   │
│  rejeción, que es lo que hace que todo esto sea      │
│  jerárquico y no una mera etiqueta:                  │
│                                                      │
│   DEFAULT            →  10.0.0.9        eth0   (al AS vecino)
└──────────────────────────────────────────────────────┘
```

**El formato de la entrada de tabla** (esto es lo que la cátedra corrige en los problemas de tablas):

```
Red Destino (X.Y.Z.W/N)  |  Siguiente Salto  |  Métrica  |  Interfaz

· Red Destino: SIEMPRE en notación CIDR, nunca "clase A".
· Siguiente Salto: dirección IP del router, o la interfaz si es
  un enlace punto a punto sin direccionar.
· Métrica: en jerarquía suele ser el CONTADOR DE GRUPOS atravesados,
  no los hops. Es la métrica "distance to boundary router".
```

**La regla de decisión, en pseudocode**, que es lo que el enunciado pide "implementar":

```
/* ============================================================
   Rutina de decisión de una red de datagramas con ruteo jerárquico.
   El orden de las pruebas es lo importante: primero el dominio
   propio (más específico), después los agregados, al final el
   default. Es longest-prefix match, y en jerarquía es el que hace
   que "todo lo que empieza con 10." no tape los prefijos internos.
   ============================================================ */

struct entrada {
    char  prefijo[16];      /* "10.0.0.0/8", "0.0.0.0/0"        */
    int   longitud_mascara; /* 8, 12, 16, 0                     */
    char  siguiente_salto[16];
    int   interfaz;
    int   grupos;           /* métrica: n° de dominios atravesados */
};

struct tabla {
    struct entrada T_intro[ MAX_INTRO ];   /* Prefijos del dominio  */
    int            n_intro;
    struct entrada T_agreg[ MAX_AGREG ];   /* Lo que anuncio afuera */
    int            n_agreg;
    struct entrada T_extern[ MAX_EXTERN ];  /* Superprefijos vecinos  */
    int            n_extern;
};

int rutear(struct tabla *t, char ip_destino[])
{
    mejor = { .prefijo = "0.0.0.0/0", .siguiente_salto = NULL,
              .longitud_mascara = -1 };        /* el default, SIEMPRE */

    /* 1. Dominio propio: prefijo más largo que coincida */
    for (i = 0; i < t->n_intro; i++) {
        e = &t->T_intro[i];
        if (coincide(ip_destino, e->prefijo, e->longitud_mascara))
            if (e->longitud_mascara > mejor.longitud_mascara)
                mejor = *e;
    }

    /* 2. Agregados que anuncia este dominio */
    for (i = 0; i < t->n_agreg; i++) {
        e = &t->T_agreg[i];
        if (coincide(ip_destino, e->prefijo, e->longitud_mascara))
            if (e->longitud_mascara > mejor.longitud_mascara)
                mejor = *e;
    }

    /* 3. Superprefijos de los dominios vecinos */
    for (i = 0; i < t->n_extern; i++) {
        e = &t->T_extern[i];
        if (coincide(ip_destino, e->prefijo, e->longitud_mascara))
            if (e->longitud_mascara > mejor.longitud_mascara)
                mejor = *e;
    }

    /* 4. Si no coincidió nada y no hay default: se descarta */
    if (mejor.siguiente_salto == NULL)
        return DESCARTAR;

    return enviar(mejor.interfaz, mejor.siguiente_salto, ip_destino);
}
```

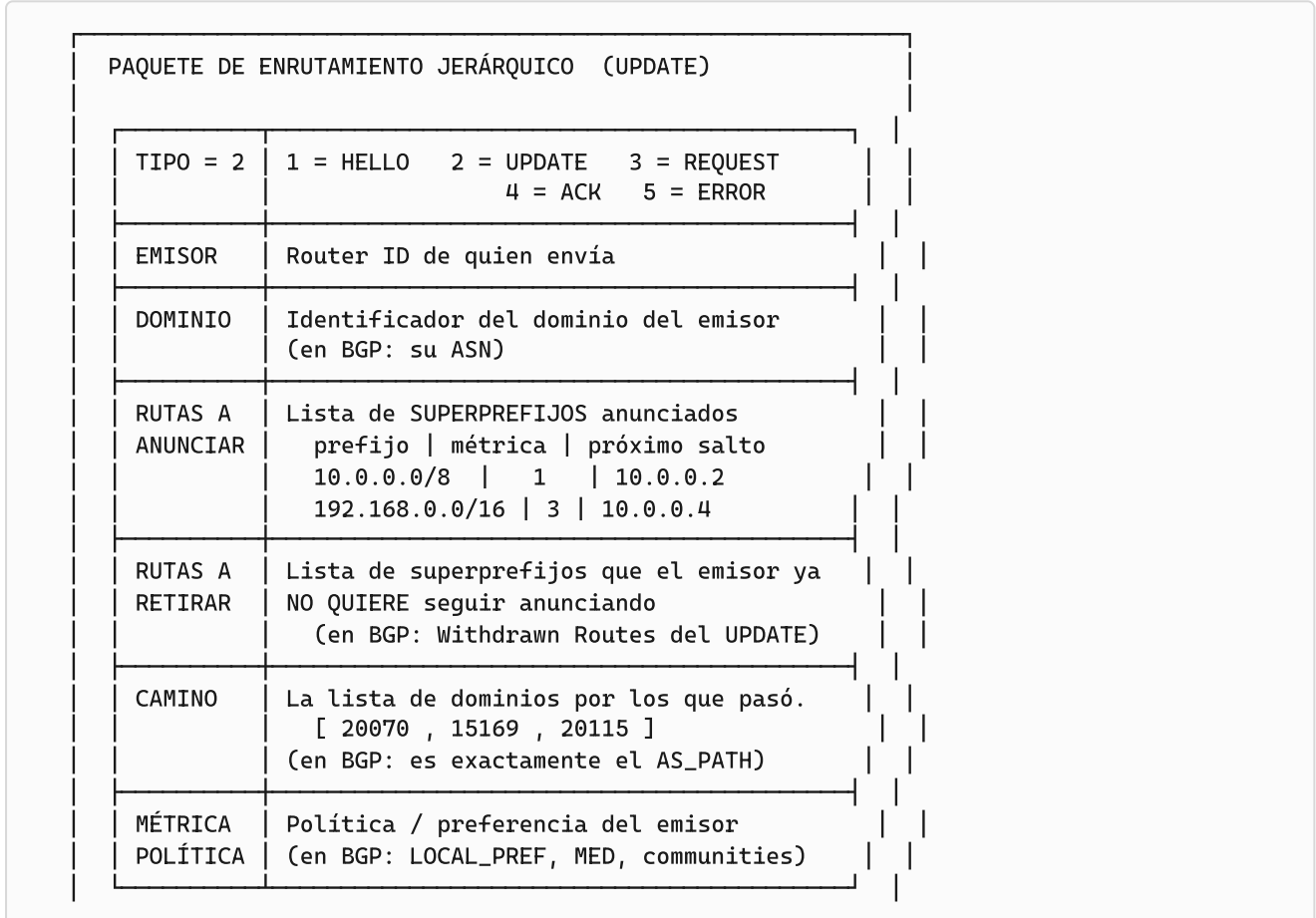
*Por qué el orden importa:* si el default ( `/0` ) se evaluara **primero**, el `longest prefix match` lo elegiría siempre, porque `0` es el prefijo más corto. El `if (e->longitud_mascara > mejor.longitud_mascara)` es lo que resuelve todo: **gana siempre el prefijo más específico**, sin importar el orden de recorrido.

### c) Los paquetes del protocolo

Un protocolo jerárquico necesita **cuatro** tipos de mensaje, y esta taxonomía es exactamente la de BGP:

| Paquete | Función | Análogo en BGP |
|---|---|---|
| **HELLO** | Descubrir vecinos y mantener la relación viva | **OPEN** + **KEEPALIVE** |
| **UPDATE** | Anunciar o retirar un **agregado** (con su métrica y el camino) | **UPDATE** |
| **REQUEST** | Pedir el estado completo de un dominio vecino, para el arranque | **UPDATE** con todas las rutas |
| **ACK / ERROR** | Confirmar o reportar un error (prefijo duplicado, incoherencia) | **NOTIFICATION** |

**El paquete UPDATE, que es el que define el protocolo**, y que hay que dibujar:




**Y la regla que cierra el circuito con el protocolo real**, y que es el corazón de §1.4:

```
Al recibir un UPDATE, el dominio receptor:

  1. RECHAZA la ruta si el CAMINO contiene su propio identificador
     de dominio.                    ← prevents bucles, exacto

  2. RECHAZA la ruta si ya tiene, para ese prefijo, una ruta cuyo
     camino sea MENOR número de dominios.   ← el "camino más corto"

  3. Si no, la ACEPTA, la preprendea con su identificador de
     dominio y la anuncia a sus vecinos.

El paso 1 es el que hace que el sistema sea ACÍCLICO por
construcción. Y es el mismo paso que BGP llama "el AS_PATH no
contiene mi ASN".
```

### d) Análisis comparativo con OSPF (el remate que suma puntos)

| | **Jerarquía tipo BGP (este problema)** | **Jerarquía tipo OSPF (áreas)** |
|---|---|---|
| Qué se publica | La **ruta agregada** | La **información de enlaces** |
| Qué se oculta | La topología interna | El camino, hasta que se calcula |
| Métrica | Número de dominios atravesados, **más política** | Costo de interfaz |
| Pérdida de información | **Ninguna**: la agregación es exacta | **Se pierde optimalidad intra-área** |
| Cálculo | No hay cálculo: se copia la decisión | Hay **SPF en cada router** |
| Escala | **Internet** (~70.000 dominios) | Red corporativa, backbone de operador |

**El remate que hay que escribir para cerrar los 30 puntos:** "*El jerarquizado por agregación no pierde información: si* `10.0.0.0/8` *está detrás de* `10.0.0.2`*, el camino más corto a cualquier prefijo de ese bloque es exactamente el mismo. Por eso el protocolo jerárquico no necesita calcular nada: el cálculo se hizo una vez, en el borde, y el resultado es correcto para todo el bloque. El jerarquizado por ocultamiento de información, en cambio, obliga a recalcular en cada nodo y acepta perder optimalidad a cambio de escalabilidad. Son dos respuestas al mismo problema con compromisos opuestos.*"

## 3.2 Examen 2007-02, Problema 2 — El enrutamiento hacia Internet con ruta por defecto

**Fuente:** Examen `eirc0702`, julio de 2007, apartado e) iii. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0702.pdf **Texto del examen (apartado e):**

> iii) El enrutamiento hacia Internet se realiza utilizando "rutas por defecto". Indique qué entrada/s debe agregar a cada tabla de enrutamiento de cada router.

⚠ **Nota:** la parte numérica depende de la figura de la red corporativa, que no se reproduce acá; lo que se resuelve es la **estructura** de la entrada de ruta por defecto y su relación con BGP, que es lo

> que la pregunta realmente apunta. Es *(solución propia)* para la parte estructural.

**a) La forma de la entrada.** En la notación de la cátedra:

```
Red Destino             Siguiente Salto    Métrica    Interfaz
─────────────────────────────────────────────────────────────
0.0.0.0/0               <ISP>              <saltos>    <puerto>
```

**Por qué `0.0.0.0/0` y no "default":** es el prefijo que coincide con **todas** las direcciones, con longitud de máscara **0**. Como el forwarding es *longest prefix match*, `0.0.0.0/0` es el prefijo **menos específico que existe**: gana contra cualquier otro prefijo, siempre que ningún otro coincida. Por eso se la pone **al final** de la tabla.

**b) Dónde va cada entrada.** La respuesta completa a "qué entrada debe agregar en cada router" es:

| Router | Entrada | Justificación |
|---|---|---|
| **R1** (el que tiene la interfaz al ISP) | `0.0.0.0/0 → ISP, métrica 1` | Es el único que ve al ISP, y ve al ISP **directamente**: 1 hop |
| **Todos los demás** | `0.0.0.0/0 → R1, métrica 2` (o la que corresponda) | No ven al ISP: lo alcanzan vía R1 |

*Justificación:* la métrica es la **cantidad de saltos** hasta el router de borde, porque para ir a Internet **todos los caminos sirven igual**. No hay ningún destino específico que requiera un camino mejor. Ese es el sentido de una **ruta por defecto**: no afirma que exista un camino óptimo, afirma que **no importa cuál**.

**c) El error clásico que hay que corregir.** Un router **no** puede tener una ruta por defecto que apunte a sí mismo. Si `R1` tuviera `0.0.0.0/0 → R1`, un paquete con destino desconocido se le mandaría a sí mismo, loop **infijo**, y como la ruta por defecto nunca expira, **no se detectaría nunca**. Por eso el `Siguiente Salto` de un default tiene que ser **un router alcanzable por un camino no por defecto**.

**d) Y cómo se relaciona con BGP — que es el puente hacia esta clase.**

```
El ejemplo (e) es "ruteo hacia Internet con ruta por defecto".
La versión real de BGP hace lo mismo pero CON POLÍTICA:

  ┌──────────────────────────────────────────────────────┐
  │ El router de borde NO anuncia "0.0.0.0/0 a todos".   │
  │ Annuncia 0.0.0.0/0 a los clientes y solo a los peers,│
  │ y NO lo anuncia a los proveedores: "no te doy tránsito│
  │ por mí, andá a buscarlo".                            │
  │                                                      │
  │ Esa decisión se toma con LOCAL_PREF, con communities │
  │ (NO_EXPORT) y con la selección de DEFAULT ROUTE.     │
  └──────────────────────────────────────────────────────┘

O sea: la ruta por defecto es el caso DEGRADADO del proceso de
decisión de BGP. En BGP, 0.0.0.0/0 es "la última entrada", y se
aplica a todo lo que no ForwardMatcheó con una ruta más específica.
Lo que cambia no es el mecanismo: es que hay ~70.000 defaults
simultáneos y cada uno tiene SU política.
```

## 3.3 ⭐ Examen 2009-03, Pregunta 2 (10 puntos) — jerarquía, IGP con jerarquías y *hot-potato*

> **Fuente:** Examen `eirc0903` / `sirc0903` — 20 de febrero de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0903.pdf **Texto del examen:**
>
> > *a) Explique el concepto de enrutamiento jerárquico. b) Describa un protocolo de enrutamiento interno (IGP) que utiliza jerarquías. c) ¿En qué consiste el mecanismo de "hot-potato routing"?*
>
> *(La solución publicada en `sirc0903` remite a las clases de teórico de Capa de Red para los tres apartados. Es el problema de BGP del corpus de FING, y vale 10 puntos.)*

**a) Enrutamiento jerárquico.** El §3.1 lo desarrolla con `2001-08 P2`. En BGP la jerarquía son los **AS con superprefijos agregados**: en vez de anunciar `10.1.2.0/24` y `10.1.3.0/24` por separado, se anuncia **un solo superprefijo** `10.0.0.0/8`, y los routers externos **no necesitan conocer la topología interna**. La optimalidad global **no se pierde**, porque la agregación es correcta por definición.

⚠️ **El contraste que hay que dar** (es lo que separa un 6 de un 10): OSPF jerarquiza **para escalar el cálculo** y BGP jerarquiza **para escalar el anuncio**. Los dos resuelven el mismo problema con herramientas **opuestas**: OSPF publica los **hechos** y esconde el camino; BGP publica el **camino** y esconde los hechos.

**b) Un IGP que utiliza jerarquías.** La respuesta de la cátedra es **OSPF** (`RFC 2328`), y su jerarquía son las **áreas**: un **ABR** (*Area Border Router*) anuncia **LSA de tipo 3** (summary) y **de tipo 4** hacia las otras áreas, y esos LSA **no se inundan** dentro del área destino. Por eso un cambio en un área **no obliga a recalcular** en las otras. Alternativa: **IS-IS**, con *Level 1* (dentro del área) y *Level 2* (el backbone).

⚠️ **El detalle que liga con la parte a):** el IGP jerarquiza **por dentro** del AS; el EGP jerarquiza **por fuera**. Son dos jerarquías **perpendiculares**, y por eso en un operador hay que configurar las dos.

**c) Hot-potato routing.** El §1.8 lo desarrolla con números. La respuesta de examen:

> **Un router prefiere reenviar el tráfico por el vecino eBGP cuya métrica IGP interna sea MENOR**, aunque el camino total sea más largo. Es un mecanismo de **pasame la papa caliente**: el router quiere **quedarse con el paquete el menor tiempo posible**. Se implementa en el **paso 7 de la tabla de decisión** (la métrica IGP hacia el `NEXT_HOP`), es decir, **el último antes del Router ID**.

⚠️ **Los tres puntos que hay que sumar:**

1. **Es el ÚLTIMO desempate, no el primero.** Precedido por `WEIGHT`, `LOCAL_PREF`, origen local, longitud de `AS_PATH`, `ORIGIN`, `MED` y eBGP-vs-iBGP. Un AS **nunca** prefiere "la ruta más cercana en hops" por sobre "la de menos ASes de por medio".
2. **Se puede anular con** `MED` (dentro del mismo AS vecino) o con **`LOCAL_PREF` / `COMMUNITY`** (entre vecinos distintos). La tabla de §2.1 lo demuestra numéricamente.
3. **Por qué existe:** el router quiere **sacarse el paquete de encima** lo más rápido posible, porque el costo interno de su red es gratis para él y el del vecino es problema del vecino. El hot-potato es la

traducción de ese comportamiento egoísta a la tabla de decisión.

## 3.4 Nota: qué hay de BGP en el corpus de exámenes

Verificado sobre todos los archivos de `material-oficial/examenes/txt`: `sirc0903` **P2 (10 pts)** es la pregunta de BGP del corpus (§3.3); `eirc0108` **Problema 2 (30 pts)** es el de **enrutamiento jerárquico**, la otra mitad de BGP (§3.1); `eirc0702` **Problema 2, apartado e** es enrutamiento hacia Internet con **rutas por defecto** (§1.2 y §1.5); y `sirc0208` / `sirc0812` la mencionan de pasada.

**Qué preguntar en el examen entonces**, y cómo se responde:

| Si el enunciado dice... | La respuesta que suma puntos |
|---|---|
| "explique el enrutamiento entre redes" | **Autonomous System** + agregación de prefijos + **política** + **path-vector**. Es el de `2001-08 P2`. |
| "explique por qué Internet escala" | Porque está **jerarquizado** en ~70.000 ASes que anuncian **agregados**, no rutas individuales. §3.1. |
| "explique el hot-potato routing" | Salir por el eBGP de **menor métrica IGP**; es el **paso 7** (último antes del Router ID). §1.8 y §3.3. |
| "diferencie RIP, OSPF y BGP" | La tabla de §1.11, con las columnas **qué viaja**, **qué algoritmo**, **qué transporte**, **qué escala**. |
| "qué es un sistema autónomo" | Conjunto de prefijos + routers con **una única política de ruteo** y un **identificador (ASN)**. |
| "por qué BGP no tiene conteo a infinito" | Porque lo que viaja es un **registro de hechos** ( `AS_PATH` ), no una **estimación derivada**. Rechazo exacto, no heurístico. §1.4. |
| "cómo se protege BGP" | **No cifra ni autentica.** `RFC 2385` (MD5) es **legado**; hoy el mecanismo de referencia son los **roles BGP de** `RFC 9234`; **RPKI/ROA** ( `RFC 6810` / `RFC 6811` ) valida el **origen del prefijo**. Y el **route leak** es el fallo más común. §1.10. |

# PARTE 4 — CIERRE

## 📌 Resumen en 10 líneas

1. **Internet es una red de redes con ~70.000 sistemas autónomos** (medilo vivo), y no hay un grafo único: hace falta **política**, no solo camino más corto.
2. El **AS** es un dominio **administrativo** (una sola política de ruteo) y el **ASN** su identificador. Hoy son **32 bits** (`RFC 4893` + `RFC 6793`), con notación decimal `RFC 5396`.
3. **BGP-4 (`RFC 4271`)** corre sobre **TCP/179** y es **vecino a vecino**: no hay flooding, no hay base común, cada router decide con su propia política.
4. **Path-vector** significa que lo que viaja es el `AS_PATH`, no una distancia. La regla es una: **no aceptar una ruta cuyo AS_PATH contenga mi propio ASN**.
5. Eso hace que BGP **no cuente a infinito y no necesite split horizon**: la detección es **exacta**, no probabilística. Pero **sí** propaga errores de configuración a toda la Internet.
6. El **handshake** es `OPEN` → `OPEN` + `KEEPALIVE` → `UPDATE` → `KEEPALIVE` cada 60 s, con hold time de 180 s, a través de los estados `Idle`, `Connect`, `Active`, `OpenSent`, `OpenConfirm`, `Established`.
7. El **proceso de decisión** desempata en orden fijo: `WEIGHT` → `LOCAL_PREF` → origen local → longitud de `AS_PATH` → `ORIGIN` → `MED` → eBGP/iBGP → métrica IGP al `NEXT_HOP` → Router ID.
8. La regla **"no re-anuncio a iBGP lo aprendido de un iBGP"** obliga al **full-mesh** (`N(N-1)/2` sesiones). **Route Reflection (`RFC 4456`)** lo baja a `O(N)`, y se protege con `ORIGINATOR_ID` y `CLUSTER_LIST`.
9. **"Hot potato"**: un AS sale por el vecino más cercano porque el último tramo lo paga él. `MED` (menor gana, comparable **solo entre rutas del mismo AS vecino**) y `LOCAL_PREF` son las dos herramientas.
10. **BGP no cifra ni autentica.** MD5 (`RFC 2385`) es legado; hoy son los **roles BGP de `RFC 9234`**; **RPKI/ROA** (`RFC 6810` / `RFC 6811`) valida el origen del prefijo. El fallo más común es el **route leak**.

## ✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

**1.** ¿Por qué en el nivel inter-AS no se puede correr Dijkstra?

<details>
<summary>Respuesta</summary>
</details>

**2.** ¿Qué es un Sistema Autónomo y qué lo define?

<details>
<summary>Respuesta</summary>
</details>

**3.** Explique por qué BGP no cuenta a infinito, y por qué eso es cualitativamente diferente de las soluciones de distance-vector.

<details>
<summary>Respuesta</summary>
</details>

**4.** Escribí el orden completo de los atributos de decisión de BGP.

▶ Respuesta

**5.** Explique el "hot potato" y cómo lo corrigen `MED` y `LOCAL_PREF`.

▶ Respuesta

**6.** ¿Por qué BGP necesita full-mesh iBGP, y qué rompe Route Reflection?

▶ Respuesta

**7.** ¿Por qué `MED` solo se compara entre rutas del mismo AS vecino?

▶ Respuesta

**8.** ¿Por qué BGP es inseguro y qué hace cada mecanismo de seguridad?

▶ Respuesta

## 🔗 Referencias

- **Examen** `eirc0108` (20-ago-2001) — Problema 2: *implementar un algoritmo de ruteo jerárquico para una red de datagramas; definir la estructura de las tablas de ruteo y los paquetes utilizados*. Es la idea de fondo de BGP. ⚠ El PDF **no publica solución**; §3.1 es *(solución propia)*.
- **Examen** `eirc0702` (jul-2007) — Problema 2, apartado e: enrutamiento hacia Internet con **rutas por defecto**.
- `eirc0903` **/** `sirc0903` — Pregunta 2 (10 pts): enrutamiento jerárquico, IGP que utiliza jerarquías (OSPF) y **hot-potato routing**. Resuelto en §3.3.
- **Nota:** el corpus no tiene un problema *de programación* sobre BGP, pero sí las preguntas teóricas de `2009-03 P2` y los dos problemas de jerarquías (`2001-08 P2`, `eirc0702 P2e`). Verificado sobre `material-oficial/examenes/txt`.
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 §5.6 "Algorithms for routing": §5.6.4 "BGP: The Border Gateway Protocol" y el *case study* de enrutamiento entre backbone de Internet.
- `RFC 4271` — BGP-4: los cuatro tipos de mensaje, la máquina de estados, el proceso de decisión y el `AS_PATH`.
- `RFC 4456` — BGP Route Reflection: `ORIGINATOR_ID`, `CLUSTER_LIST`, reglas de no reenvío.
- `RFC 1997` — BGP Communities (`NO_EXPORT`, `NO_ADVERTISE`, `NO_EXPORT_SUBCONFED`) y el formato `AS:value`.
- `RFC 4893` — el problema del AS de 2 octetos · `RFC 6793` — el Capability de 4 octetos · `RFC 5396` — notación decimal `asdot` / `asplain`.
- `RFC 2385` — MD5 para BGP: **el mecanismo clásico, hoy legado** · `RFC 9234` (2022) — **roles BGP**, reemplazo de referencia.
- `RFC 6810` — RTR (RPKI to Router) · `RFC 6811` — BGP Prefix Origin Validation (ROA).
- `RFC 7854` — BMP, monitoreo de BGP desde fuera.

- `RFC 9552` — BGP-LS (dic-2023), obsoleta `RFC 7752` · `RFC 7432` — EVPN · `RFC 7348` — VXLAN · `RFC 8402`, `RFC 8660`, `RFC 8986` — Segment Routing / SRv6.
- **Clase anterior:** 33 — Enrutamiento link-state, Dijkstra y OSPF
- **Clase siguiente:** 35 — Fragmentación, MTU y path MTU